Installation and Code Structure Guide: Using AI to Improve Supply Chains in Oregon

A small farm logistics cost modeling application.

David Foster – Caden Smith – Luke Garci

CS 463 Spring 2026 @ Oregon State University

Table of Contents

Hosted Project Link:

https://Food.Opliva.com

Credentials for Testing:

Username: Test

Password: 1234567890

Project GitHub:

Main Repo: https://github.com/Cadensmith1123/AI-Improve-Supply-Chains-Oregon

Release: https://github.com/Cadensmith1123/AI-Improve-Supply-Chains-Oregon/releases/tag/v1.0.0

Released Branch: https://github.com/Cadensmith1123/AI-Improve-Supply-Chains-Oregon/tree/freeze/testing-finalizing

Installation on a New Server:

Setting up .env file

This project requires a .env file to run. The following are the required fields and what they are used for.

Database variables for the main database:

- DB_USER: The database username
- DB_PASSWORD: The database password for that user
- DB_HOST: The host address for the database
- DB_PORT: The database port
- DB_NAME: the name of the database

Database variables for the authentication database:

- AUTH_DB_USER: The database username
- AUTH_DB_PASSWORD: The database password for that user
- AUTH_DB_HOST: The host address for the database
- AUTH_DB_PORT: The database port
- AUTH_DB_NAME: The database name

Security variables:

- JWT_SECRET: A long string of characters for signing and verifying the JavaScript web token.
- SECRET_KEY: Flask's secret key, used to sign the password reset token and the anonymous recovery key.
- JWT_ISSUER: Used to verify that tokens were minted by this application (added for later extensibility)
- JWT_AUDIENCE: Used to verify what service a JWT token is meant for (added for later extensibility)
- JWT_ACCESS_TTL_SECONDS: The length of time a JavaScript web token is valid for before it expires (should be set to something like 3600)
- ANON_RECOVERY_TTL_SECONDS: How many seconds an anonymous account stays active between application interactions (Set to 604800 for 7 day access)
- MAPBOX_TOKEN: The API key for a MapBox account for getting distances between origin and destination and showing a map on the route details page.

Required Dependencies:

- Python 3.9+ environment with the following packages installed
    - PyJWT
    - Pandas
    - Flask
    - Gunicorn
    - Python-dotenv
    - Mysql-connector-python
    - Pyotp
    - Itsdangerous
    - Qrcode[pil]
- These can be installed using the following command from the root directory of the project: pip install -r requirements.txt

Installation Procedure:

Clone the Repo and create a virtual environment

Run the following commands:

1. git clone https://github.com/Cadensmith1123/AI-Improve-Supply-Chains-Oregon.git
2. cd AI-Improve-Supply-Chains-Oregon

3. python -m venv .venv
4. Activate the environment with one of the following commands
    a. Windows: .venv\Scripts\Activate.ps1
    b. maxOS/Linux: source .venv/bin/activate

Install Dependencies

- Run: pip install -r requirements.txt

Configure Environment

- Move add the .env file defined by the “Setting up .env file” section above

Set up the database (database must be active and reachable using the settings in .env)

- python db/functions/rebuild_database.py
- NOTE: The first account created using the UI will be prepopulated with generic data as a test account.

Setup anonymous user cleanup.

- This is environment dependent for Linux and MacOS run the following:
    - 0 3 * * * cd /path/to/AI-Improve-Supply-Chains-Oregon && /path/to/AI-Improve-Supply-Chains-Oregon/.venv/bin/python -m auth.cleanup_anonymous >> /var/log/anon_cleanup.log 2>&1
        - This will log the deletions inside /var/log/anon_cleanup.log
- For running on Windows you must use Task Scheduler with the following settings
    - Program: C:\path\to\repo\.venv\Scripts\python.exe
    - Arguments: -m auth.cleanup_anonymous
    - Start in: C:\path\to\repo
    - Trigger: Daily, 3:00AM

Run the app (gunicorn)

- cd frontend_flask
- gunicorn -w 4 -b <IP>:<port> app:app
    - For running locally use 127.0.0.1 for the IP and 5000 for the port
    - For running with a reverse proxy (e.g. nginx) to expose to the internet use the appropriate IP and port for your reverse proxy setup

Run the app (Flask native)

- cd frontend_flask

- python app.py

Run tests (database must be active and reachable using the settings in .env)

- pytest tests/

Code Structure and Where things live

This section will provide an overview of where different files live and the purpose of the file structure of this project

Base:

- auth : contains python files for the authorization which works independently from the main application
    - auth_demos: contains demo scripts for manual authentication smoke tests
    - __init__.py: blank init for package addressing in other modules
    - cleanup_anonymous.py: file for cleaning up anonymous users after their account expires
    - middleware.py: file that installs and manages the JWT middleware
    - passwords.py: file that is used for encrypting and decrypting user passwords
    - style.css: styling for totp authentication page
    - tokens.py: mints and verifies JWTs
    - totp.py: manages TOTP authentication assets
    - user_management: functions that manage users and call authentication procedures on the authentication database.
- db: holds files that access the main database and DML and DDL .sql files for setting up the database
    - demos: holds demo functions for manual database smoke tests
    - functions: holds all functions that directly touch the database or call a database procedure
        - simple_functions: holds files that call a database procedure directly
            - create.py: holds functions that create records on the database
            - delete.py: holds functions that delete records on the database
            - read.py: holds functions that read and return records on the database
            - update.py: holds functions that update records on the database

        - tenant_functions: holds files that get the users “tenant_id” and wrap the simple_functions and inject a users tenant ID to return that users data only
            - scoped_create.py: wraps the create.py functions
            - scoped_delete.py: wraps the delete.py functions
            - scoped_read.py: wraps the read.py functions
            - scoped_update.py: wraps the update.py functions
        - connect.py: holds functions that return connections to the main database and the auth database
        - rebuild_database.py: holds the code to setup the database structures and install the database procedures
        - scenario_management.py: holds functions that call the tenant functions to produce more complex results.
        - procedures: holds all the procedure definitions as .sql files for installation on the database.
        - schema: holds the DDL files for both the auth database and the main database.
- Docs: holds important docs and ERD images
- frontend_flask: Holds the main application code
    - static: holds static files
        - quick-create.js: used to allow creation of assets from other assets overlays (the “+” button next to the dropdown menus)
        - style.css: the main styling for the web app
    - templates: holds the .html templates that define the user interface
    - access_db.py: the connection point for all database interactions. Calls functions in “tenant_functions” folder and “scenario_management.py”
    - app.py: the entry point for flask or gunicorn to run
    - assets_bp.py: the blueprints for adding, editing, or deleting assets
    - auth_bp.py: the blueprints for sign in, registration, and TOTP authentication
    - csv_template_bp.py: the blueprint for CSV templates for each asset type
    - depreciation_insurance.py: functions used to calculate depreciation, insurance and maintenance cost
    - drivers_import_bp.py: the blueprint for uploading driver CSVs
    - location_import_bp.py: the blueprint for uploading location CSVs
    - logic.py: The location of pure calculation functions
    - map_route.py: the blueprint for showing the map on route details
    - products_bp.py: the blueprint for managing product listings
    - products_import_bp.py: the blueprint for uploading product CSVs

    - routes_bp.py: the blueprint for the main routes dashboard
    - routes_import_bp.py: the blueprint for uploading route CSVs
    - vehicles_import_bp.py: the blueprint for uploading vehicle CSVs
- tests: holds the test scripts and test suite
    - auth: holds test scripts for testing anonymous mode, JWT tokens, and TOTP authentication
    - db_tests: holds tests used for testing the database is fully set up and database functions work correctly
    - security: tests that endpoints are protected, no data leakage or access violations, and JWT expiry.
- .env: the environment file
- pyproject.toml: used to help pytest know where test files exist
- requirements.txt: used to install dependencies